Guía de repaso I2

IIC2413, Bases de Datos
Benjamín Saavedra

Convenciones

- **Dependencias.** X^+ es la clausura de X según $\mathcal{F}$; F_S son las dependencias de S según la clase 8 (las $\bar{X} \to \bar{Y}$ con $\mathcal{F} \models \bar{X} \to \bar{Y}$ y $\bar{X}, \bar{Y} \subseteq \text{att}(S)$). Un atributo es *primario* si está en alguna llave. En `toBCNF` y en el algoritmo de descomposición en 3NF se usa la versión de la clase 9 y la de la clase 10, respectivamente.
- **B+tree de orden d.** Hojas con entre d y $2d$ datos, directorios (salvo la raíz) con entre d y $2d$ separadores. Split de hoja: d datos a la izquierda y $d+1$ a la derecha, y el separador se *copia* al padre; split de directorio: el separador *sube*. Se inserta sin redistribución salvo que se indique. En la eliminación se intenta redistribuir con el vecino izquierdo, luego con el derecho, y si no se puede se hace unsplit (con el izquierdo si existe). Solo se usan vecinos con el mismo padre. Los separadores no se modifican al eliminar un dato sin rebalancear.
- **Buffer manager.** `usar(X)` abrevia `pin(X)` seguido de `unpin(X, false)`. **FIFO** saca la página (con `#pin` $= 0$) que lleva más tiempo cargada; **LRU**, la de uso más antiguo.
- **Hashing.** $h_n(k)$ son los últimos n bits de k. Extendable hashing parte con profundidad global 0 y un bucket vacío. Si tras un split el dato sigue sin caber, se vuelve a aplicar el algoritmo de inserción.
- **Operadores con y sin duplicados.** Como en la clase 16. Si t aparece r veces en R y s veces en S, aparece $r+s$ veces en $R \cup S$ (como `UNION ALL`), mín(r,s) en $R \cap S$ y máx$(r-s, 0)$ en $R \setminus S$ (como `EXCEPT ALL`).
- **Costos.** Se miden en páginas leídas o escritas, según el ejercicio.

1. Dependencias funcionales y llaves

Ejercicio 1.

Sea $\mathbf{R}(A, B, C, D, E)$ con $\mathcal{F} = \{A \to B,\ A \to C,\ B \to E,\ CE \to A,\ CE \to D\}$.

(a) Encuentre **todas** las llaves de $\mathbf{R}$ calculando clausuras, y justifique que no hay otras sin revisar los 2^5 subconjuntos.

(b) ¿Cuántas superllaves tiene $\mathbf{R}$? Cuéntelas sin enumerarlas todas.

(c) ¿Qué atributos son primarios?

(d) ¿Está $\mathbf{R}$ en BCNF? ¿Y en 3NF? Indique las violaciones de BCNF con un solo atributo a la derecha.

(e) Aplique `toBCNF` partiendo por $B \to E$. ¿La descomposición preserva las dependencias? Si no, indique cuáles se pierden.

(f) *Problema inverso.* Para $\mathbf{S}(A, B, C, D)$, dé un $\mathcal{F}$ cuyas únicas llaves sean exactamente:

(i) $\{A\}$ y $\{B, C\}$;
(ii) $\{A, B\}$, $\{A, C\}$ y $\{B, C\}$.

Explique por qué ningún $\mathcal{F}$ sobre $\mathbf{S}$ tiene como únicas llaves $\{A\}$ y $\{A, B\}$. Dé un $\mathcal{F}$ sobre $\mathbf{S}$ con seis llaves.

(g) Para cada afirmación sobre una relación $\mathbf{T}$ cualquiera con dependencias $\mathcal{F}$, demuéstrela o dé un contraejemplo:

(i) Dos llaves de $\mathbf{T}$ pueden tener tamaños distintos.
(ii) Si $\mathbf{T}$ tiene una única llave, entonces $\mathbf{T}$ está en BCNF.
(iii) Si todas las llaves de $\mathbf{T}$ tienen un solo atributo, entonces $\mathbf{T}$ está en BCNF.

Ejercicio 2.

Vuelos(vueloId, fecha, origen, destino, aerolinea, duracion), con la instancia:

vueloId	fecha	origen	destino	aerolinea	duracion
LA800	2026-09-01	SCL	MIA	LATAM	480
LA800	2026-09-08	SCL	MIA	LATAM	480
LA801	2026-09-01	MIA	SCL	LATAM	455
AA100	2026-09-01	SCL	MIA	American	470
AA100	2026-09-15	SCL	MIA	American	470

(a) Para cada dependencia, indique si la instancia la satisface y, si no, dos tuplas que la violan: (i) vueloId → origen destino aerolinea duracion; (ii) vueloId fecha → duracion; (iii) origen destino → duracion; (iv) origen destino aerolinea → duracion; (v) fecha aerolinea → vueloId.

(b) (iv) se cumple en la instancia. ¿Se puede concluir que **Vuelos** la satisface siempre (para toda instancia válida del dominio)? ¿Y de (iii) se puede concluir que *no* se satisface siempre? Explique la asimetría y dé un supuesto del mundo real que haga razonable (ii) pero no (v).

(c) Demuestre, desde la definición de $\models$ (sin el algoritmo de clausura), que si $\mathbf{R} \models X \to Y$ y $\mathbf{R} \models Y \to Z$ entonces $\mathbf{R} \models X \to Z$. Use el mismo estilo de argumento para decidir si $\mathbf{R} \models X \to Y$ y $\mathbf{R} \models XY \to Z$ implican $\mathbf{R} \models X \to Z$.

(d) **Empleado**(rut, nombre, depto, jefe) con $\mathcal{F} = \{\mathsf{rut} \to \mathsf{nombre\,depto\,jefe}\}$. ¿Es {rut} llave? ¿Es {rut, nombre} superllave? ¿Es {nombre} superllave? Para la última, dé una instancia que satisfaga $\mathcal{F}$ y muestre que la respuesta no puede ser "sí"; ¿cambia algo si el dominio dice que no hay dos empleados con el mismo nombre?

2. Clausura, razonamiento y minimal

Ejercicio 3.

Sea $\mathbf{T}(A, B, C, D, E)$ con $\mathcal{F} = \{A \to BC,\ CD \to E,\ B \to D,\ E \to A\}$.

(a) Calcule con el algoritmo de clausura A^+, B^+, $(CD)^+$, E^+, D^+ y $(BC)^+$.

(b) Con las clausuras de (a) u otras, decida si $\mathcal{F} \models B \to A$, $\mathcal{F} \models CD \to A$, $\mathcal{F} \models B \to E$ y $\mathcal{F} \models D \to C$.

(c) Encuentre todas las llaves de $\mathbf{T}$.

(d) Encuentre un conjunto minimal equivalente a $\mathcal{F}$ aplicando en orden: separar lados derechos, eliminar atributos redundantes de lados izquierdos, eliminar dependencias redundantes. Muestre el cálculo de clausura de cada eliminación.

(e) ¿Está $\mathbf{T}$ en BCNF? ¿En 3NF?

(f) Aplique `toBCNF` y el algoritmo de descomposición en 3NF a $\mathbf{T}$. Compare ambos resultados: ¿cuál preserva las dependencias?

3. Descomposiciones: pérdida de información y preservación

Ejercicio 4.

Sea $\mathbf{R}(A, B, C, D)$ con $\mathcal{F} = \{A \to B,\ C \to D\}$. Una descomposición de $\mathbf{R}$ en $\bar{X}$ e $\bar{Y}$ es *sin pérdida* si $\mathbf{R} = \pi_{\bar{X}}(\mathbf{R}) \bowtie \pi_{\bar{Y}}(\mathbf{R})$ para toda instancia $\mathbf{R}$ que satisface $\mathcal{F}$.

(a) Para cada descomposición decida si es sin pérdida. Si lo es, justifique; si no, dé una instancia de $\mathbf{R}$ que satisfaga $\mathcal{F}$ y cuyo join de las proyecciones tenga más tuplas que $\mathbf{R}$: (i) $\bar{X} = \{A, B\}$, $\bar{Y} = \{A, C, D\}$; (ii) $\bar{X} = \{A, B\}$, $\bar{Y} = \{B, C, D\}$; (iii) $\bar{X} = \{A, B\}$, $\bar{Y} = \{C, D\}$.

(b) Sea $\bar{Z} = \bar{X} \cap \bar{Y}$. Demuestre que si $\mathcal{F} \models \bar{Z} \to \bar{X}$ (o $\mathcal{F} \models \bar{Z} \to \bar{Y}$), la descomposición es sin pérdida. Explique por qué esto garantiza que cada paso de `toBCNF` es sin pérdida.

(c) Demuestre el recíproco: si $\mathcal{F} \not\models \bar{Z} \to \bar{X}$ y $\mathcal{F} \not\models \bar{Z} \to \bar{Y}$, existe una instancia que satisface $\mathcal{F}$ para la cual la descomposición pierde información. *Hint:* use una instancia con dos tuplas que coinciden exactamente en $\bar{Z}^+$.

(d) Decida, con demostración o contraejemplo, si son verdaderas: (i) toda descomposición sin pérdida preserva las dependencias; (ii) toda descomposición que preserva las dependencias es sin pérdida.

Ejercicio 5.

Un catálogo registra **Estudios**(titulo, estudio, direccion) con $\mathcal{F} = \{\text{titulo} \to \text{estudio},\ \text{estudio} \to \text{direccion}\}$ (cada título lo produce un estudio, y cada estudio tiene una dirección).

(a) Encuentre las llaves de **Estudios** y diga si está en BCNF y en 3NF.

(b) Considere D_1: {titulo, estudio} y {titulo, direccion}. Calcule F_S y F_T (clase 8), y decida si D_1 es sin pérdida y si preserva las dependencias. Si no preserva, dé instancias S' y T' que satisfagan F_S y F_T cuyo join viole $\mathcal{F}$.

(c) Haga lo mismo con D_2: {titulo, estudio} y {estudio, direccion}. ¿Es la que produce `toBCNF`?

(d) Ambas partes de D_1 están en BCNF, y D_1 es sin pérdida; sin embargo es una mala descomposición. ¿Qué anomalía reaparece? Dé una modificación de S' o T' que cada tabla acepte pero que deje inconsistente la información.

(e) Para $\mathbf{R}(A, B, C)$ con $\mathcal{F} = \{A \to B,\ B \to C\}$ hay tres descomposiciones binarias posibles: $\{AB, AC\}$, $\{AB, BC\}$ y $\{AC, BC\}$. Clasifique cada una según si es sin pérdida y si preserva las dependencias. Relacione su respuesta con la forma de la dependencia que se pierde.

4. Forma normal de Boyce-Codd

Ejercicio 6.

(a) *Catálogo.* **Catalogo**(libro, autor, editorial, pais) con $\mathcal{F} = \{\text{libro} \to \text{autor},\ \text{autor} \to \text{editorial},\ \text{editorial} \to \text{pais}\}$.

(i) Encuentre la llave e indique todas las violaciones de BCNF (considere también las que se derivan de $\mathcal{F}$).

(ii) Aplique `toBCNF` eligiendo primero autor $\to$ editorial. En cada paso indique X, X^+ y X^-.

(iii) Repita eligiendo primero editorial $\to$ pais. ¿Cambia el resultado?

(iv) ¿Es sin pérdida y preserva las dependencias?

(b) *Boletas.* Una tienda registra una fila por cada producto de cada boleta en **LineaCompra**, con los atributos idBoleta, fechaBoleta, rutCliente, nombreCliente, sku, nombreProducto, precioUnitario y cantidad. Cada boleta tiene una fecha y un cliente fijos; cada cliente tiene un nombre fijo; cada producto tiene un nombre y un precio fijos; y cada línea indica cuántas unidades de un producto se compraron en una boleta.

idBoleta	fechaBoleta	rutCliente	nombreCliente	sku	nombreProducto	precioUnitario	cantidad
1001	2026-09-01	C1	Ana Torres	P1	Tornillo	50	10
1001	2026-09-01	C1	Ana Torres	P2	Martillo	4990	1
1002	2026-09-02	C2	Luis Vidal	P1	Tornillo	50	5
1003	2026-09-02	C1	Ana Torres	P3	Pintura	8990	2

(i) Escriba las dependencias funcionales y encuentre la llave.

(ii) ¿Está en BCNF? ¿En 3NF? Justifique.

(iii) Aplique `toBCNF`. ¿Cuántas relaciones obtiene, y con qué llaves?

(iv) Sobre la instancia dada, dé un ejemplo de anomalía de actualización, de inserción y de eliminación de la tabla original. ¿Cuáles desaparecen en su descomposición?

Ejercicio 7.

(a) Demuestre que toda relación con exactamente dos atributos está en BCNF. Analice los casos de dependencias no triviales posibles.

(b) Sea $\mathbf{R}(A, B, C, D)$ con llave única $\{A, D\}$ y $\mathcal{F} = \{A \to B,\ A \to C\}$. Aplique `toBCNF` empezando con $A \to B$, y luego empezando con $A \to C$. ¿Hay diferencia? Explique por qué depende (o no) de cuál se elija, usando que X^+ se calcula con todo $\mathcal{F}$.

(c) `toBCNF` no siempre da el mismo resultado: para $\mathbf{S}(A, B, C)$ con $\mathcal{F} = \{B \to C,\ C \to B\}$ aplique el algoritmo eligiendo $B \to C$ y luego eligiendo $C \to B$. Compare ambos resultados: ¿son sin pérdida? ¿Preservan? ¿Cuál preferiría?

(d) Demuestre que `toBCNF` termina: cada paso produce relaciones con estrictamente menos atributos.

5. Tercera forma normal

Ejercicio 8.

Un cine programa películas en **Showtimes**(cinema, city, movie) con $\mathcal{F} = \{\mathsf{cinema} \to \mathsf{city},\ \mathsf{city\ movie} \to \mathsf{cinema}\}$ (por ciudad, cada película se exhibe en a lo más un cine).

(a) Encuentre las llaves y los atributos primarios. ¿Está en BCNF? ¿En 3NF?

(b) Dé una instancia con al menos dos filas del mismo cine y describa la anomalía de actualización si el cine cambia de ciudad. ¿Por qué esa anomalía puede ocurrir en una relación en 3NF y no en una en BCNF?

(c) Descomponga con `toBCNF`. Calcule F_S y F_T, y decida si la descomposición es sin pérdida y si preserva las dependencias. ¿Qué restricción del negocio ya no se puede verificar mirando una sola tabla?

(d) Aplique el algoritmo de descomposición en 3NF. Compare con **Showtimes**: ¿qué relaciones obtiene?

(e) Complete la tabla (sí/no) para (i) **Showtimes** sin descomponer, (ii) la descomposición BCNF de (c), (iii) la descomposición 3NF de (d): sin pérdida, preserva las dependencias, y elimina la anomalía de actualización de (b).

(f) Sea $\mathbf{Q}(A, B, C, D)$ con $\mathcal{F} = \{AB \to C,\ C \to A\}$. Encuentre todas las llaves y los atributos primarios, y decida si $\mathbf{Q}$ está en BCNF y en 3NF. ¿Qué dependencia hace fallar BCNF y por qué 3NF la tolera?

(g) Demuestre: si una relación tiene una *única* llave y está en 3NF, entonces está en BCNF. (Compárelo con el caso de (a), que tiene dos llaves.) *Hint:* suponga una violación $X \to a$ con a primario y construya otra llave.

Ejercicio 9.

Para cada esquema: (i) encuentre las llaves; (ii) verifique que el conjunto $\mathcal{F}$ dado es minimal; (iii) aplique el algoritmo de descomposición en 3NF (la relación R_K incluida) y escriba las dependencias de cada relación; (iv) ¿está cada relación resultante en BCNF? (v) ¿tiene sentido agregar R_K o ya está contenida en otra?

(a) **Corredora**(inversionista, accion, corredor, oficina, cantidad, dividendo) con $\mathcal{F} = \{$accion $\rightarrow$ dividendo, inversionista $\rightarrow$ corredor, inversionista accion $\rightarrow$ cantidad, corredor $\rightarrow$ oficina$\}$.

(b) **Horario**(curso, profesor, hora, sala, estudiante, nota) con $\mathcal{F} = \{$curso $\rightarrow$ profesor, hora sala $\rightarrow$ curso, hora profesor $\rightarrow$ sala, hora estudiante $\rightarrow$ sala, curso estudiante $\rightarrow$ nota$\}$.

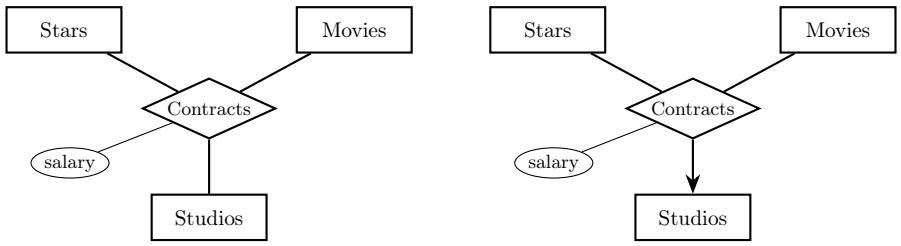
(c) Repita (1) quitando la dependencia inversionista accion $\rightarrow$ cantidad (por ejemplo, porque la cantidad ya no se registra por inversionista y acción). ¿Qué cambia en la llave y en el rol de R_K?

(d) Aplique `toBCNF` a (1) y a (2). ¿Cuál de las dos descomposiciones preserva las dependencias?

6. Diseño E/R y dependencias funcionales

Ejercicio 10.

Entidades Stars($\underline{name}$), Movies($\underline{title}, \underline{year}, length$) y Studios($\underline{name}, address$), y la relación ternaria Contracts con el atributo $salary$. Dos versiones: V1 sin flechas y V2 con una flecha hacia Studios.



La tabla de la relación tiene los atributos starName, title, year, studioName, salary.

(a) Para V1 y V2 escriba las dependencias funcionales de **Contracts**, encuentre sus llaves y diga si están en BCNF. Explique qué efecto tiene la flecha sobre las llaves.

(b) Se agrega a V2 una segunda flecha, hacia Movies (versión V3). Explique con palabras qué dice cada flecha de V3, escriba las dependencias, encuentre las llaves y decida si está en BCNF. Las llaves tienen tamaños distintos: ¿es coherente con el diagrama?

(c) Se propone traducir la relación varios-a-varios ActIn (entre Stars y Movies) absorbiéndola en la tabla de películas: **Movies**(title, year, length, starName). Escriba las dependencias, encuentre la llave y decida si está en BCNF. Dé una anomalía concreta y explique por qué la regla es "al absorber una varios-a-uno $E \rightarrow F$ se agregan a E las *llaves* de F" y solo para varios-a-uno.

(d) En la clase, la relación varios-a-uno Owns (de Movies a Studios) se tradujo como **Owns**(title, year, studioName) con llave formada por los tres atributos. ¿Es esa una llave o solo una superllave? ¿Qué dependencia expresa la flecha de Owns? ¿Qué error produce declarar los tres atributos como llave?

(e) Suponga que en el dominio el estudio de un contrato *siempre* es el que posee la película (según Owns). Escriba las dependencias de **Contracts** en V2 más esta restricción. Muestre que hay redundancia, diga si está en BCNF y qué descomposición corregiría el diseño. ¿Qué dependencia no dibuja el diagrama V2?

7. Almacenamiento y buffer manager

Ejercicio 11.

Convenciones. El disco tiene, en promedio, 6,46 ms de búsqueda (*seek*), 4,17 ms de retraso rotacional y 0,13 ms de transferencia por bloque (valores de la clase 12). Una página es un bloque de 4096 bytes. Al leer bloques contiguos, solo el primer bloque paga búsqueda y rotación; cada bloque adicional paga solo la transferencia.

(a) ¿Cuánto demora leer 1000 bloques distintos (ubicados al azar) y cuánto 1000 bloques consecutivos? ¿Cuántas veces más lento es lo primero?

(b) Una relación tiene 10^6 tuplas de tamaño fijo, de 100 bytes cada una. Cada página guarda un encabezado de 96 bytes y el resto son *slots* de 100 bytes. ¿Cuántas tuplas caben por página? ¿Cuántas páginas ocupa la relación? ¿Cuántas páginas lee un recorrido completo, y cuánto demora si las páginas son contiguas y si están ubicadas al azar? ¿Cuál es el RID de la tupla del slot 7 de la página 1001?

(c) Una página de 128 bytes guarda tuplas de tamaño variable con el esquema de la clase: un encabezado de 12 bytes, un directorio de entradas de 4 bytes (una por tupla) y las tuplas. Una entrada de directorio de una tupla eliminada queda libre y se reutiliza. Parte vacía y se hacen las operaciones: insertar t_1 (30 bytes), t_2 (20 bytes), t_3 (40 bytes); eliminar t_2; insertar t_4 (30 bytes); insertar t_5 (10 bytes). Para cada una indique si cabe, el espacio libre total y si hace falta compactar la página. ¿Cambia el RID de t_3?

(d) Una relación ocupa 1000 páginas de datos, y solo 300 de ellas tienen algo de espacio libre. Se quiere insertar una tupla. Compare el peor caso de lecturas para encontrar una página donde quepa, con tres organizaciones: (i) páginas consecutivas sin más información; (ii) lista de páginas con espacio libre, a partir de una página de encabezado (suponga que ninguna de las 300 tiene espacio suficiente para esta tupla, salvo la última de la lista); (iii) directorio de páginas con una entrada de 8 bytes por página de datos (con el espacio libre de cada una), almacenado en páginas de 4096 bytes.

Ejercicio 12.

Un buffer manager tiene **3 frames**, parte vacío y recibe las operaciones siguientes sobre las páginas A, $\ldots$, E:

1. `pin(A)`	7. `pin(A)`	13. `unpin(B, false)`
2. `unpin(A, false)`	8. `unpin(A, false)`	14. `unpin(D, false)`
3. `pin(B)`	9. `pin(D)`	15. `pin(E)`
4. `unpin(B, true)`	10. `pin(A)`	16. `unpin(E, false)`
5. `pin(C)`	11. `unpin(A, true)`	17. `pin(A)`
6. `unpin(C, false)`	12. `pin(B)`	18. `unpin(A, false)`

Una página `dirty` que sale del buffer se escribe en disco.

(a) Muestre el estado de cada frame (página, `#pin` y `dirty`) después de cada operación con **FIFO**, e indique cada lectura y escritura.

(b) Haga lo mismo con **LRU**.

(c) ¿Cuántas lecturas y escrituras hace cada política? Al terminar, ¿queda algún cambio sin escribir en disco?

(d) En la operación 12, ¿por qué ninguna de las dos políticas puede elegir D como víctima?

(e) Suponga que entre las operaciones 12 y 13 llegan `pin(C)` y, antes de cualquier `unpin`, `pin(E)`. ¿Qué ocurre con cada política?

Ejercicio 13.

Considere la secuencia de peticiones `usar` de $1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$ (el buffer parte vacío).

(a) Muestre el contenido del buffer después de cada petición y cuente las lecturas con FIFO para 3 frames y para 4 frames. ¿Qué observa?

(b) Repita (a) con LRU.

(c) Sea $S_N(t)$ el conjunto de páginas en un buffer LRU de N frames después de las primeras t peticiones de una secuencia cualquiera de `usar`. Demuestre que $S_N(t) \subseteq S_{N+1}(t)$ para todo t y concluya que con LRU agregar frames nunca aumenta las lecturas.

(d) Explique por qué el argumento de (c) no funciona para FIFO y dé la petición exacta donde falla la inclusión en el ejemplo de (a).

Ejercicio 14.

Un buffer manager tiene N frames, todos ocupados y con `#pin` $= 0$. Una *solicitud* de una página es un `pin` seguido inmediatamente de su `unpin`$(\cdot,$ `false`$)$. Una política de reemplazo es *determinista* si la víctima que elige depende solo de las solicitudes anteriores.

(a) Demuestre que para toda política determinista P existe una secuencia S de N solicitudes tal que P hace N lecturas a disco.

(b) Demuestre que, para esa misma secuencia S, existe otra política P' que hace una sola lectura.

(c) ¿Por qué el argumento de (a) no funciona si la política es aleatoria? ¿Significa que una política aleatoria es mejor en el peor caso?

(d) Con $N = 3$ y el buffer inicial $\{A, B, C\}$ (cargadas y usadas en ese orden), construya la secuencia de (a) para FIFO, para LRU y para MRU (la política que saca la página de uso más reciente), usando la página D como página fuera del buffer. Para cada una dé la política P' de (b).

8. Índices

Ejercicio 15.

Una relación **Ventas** tiene $n = 2\,000\,000$ de tuplas, 50 tuplas por página y las tuplas están ordenadas por id (que es llave) en $P = 40\,000$ páginas contiguas. Las páginas son de 4096 bytes. Un B+tree sobre id usa 4 bytes por separador, 8 por puntero y 12 por data entry (llave y puntero) en las hojas, con el orden d más grande posible (directorios con $2d$ separadores y $2d + 1$ punteros, hojas con $2d$ data entries). Una búsqueda por valor en el B+tree lee un nodo por nivel (altura H: $H + 1$ páginas hasta la hoja).

(a) ¿Cuántas páginas lee un recorrido completo? ¿Cuántas, en el peor caso, una búsqueda por valor con búsqueda binaria sobre las páginas ordenadas?

(b) Calcule el orden d y justifique que la altura del B+tree es 2 para este n (cotas de altura mínima y máxima con ese d, suponiendo que hay una data entry por tupla).

(c) Costo de la búsqueda por valor de un id con el B+tree unclustered (el índice apunta a las tuplas), y con un B+tree clustered (las hojas son las páginas de datos, así que la búsqueda termina en la página de la tupla).

(d) Una consulta por rango devuelve k tuplas consecutivas en el orden de id. Con $k = 500$, calcule el costo con: búsqueda binaria y recorrido secuencial; B+tree clustered (altura 2: lee los 2 directorios y luego $\lceil k/50 \rceil$ páginas de datos); B+tree unclustered (3 páginas hasta la primera hoja, $\lceil k/340 \rceil - 1$ hojas más y una página por tupla); y recorrido completo.

(e) ¿Para qué valores de k el B+tree unclustered cuesta menos que el recorrido completo? Use el modelo de (d).

(f) Se inserta una tupla con un id que cae en medio del archivo ordenado, sin espacio libre en las páginas. Estime las lecturas y escrituras si hay que mantener el orden en páginas consecutivas (en promedio, la mitad del archivo se corre) y compárelas con las de un B+tree (sin split).

Ejercicio 16.

Sea **Players**(<u>id</u>, name, team, age, salary) con los índices

I_1: B+tree sobre (age)	I_3: B+tree sobre (name, team)
I_2: hash sobre (age)	I_4: hash sobre (name, team)

(a) Para cada consulta indique qué índices pueden acelerarla y cuáles no, y justifique:

(I) `WHERE age = 30`

(II) `WHERE age BETWEEN 25 AND 30`

(III) `WHERE name = 'Rodri' AND team = 'Man City'`

(IV) `WHERE name = 'Rodri'`

(V) `WHERE team = 'Man City'`

(VI) `ORDER BY age`

(VII) `WHERE age <> 30`

(b) Un equipo propone eliminar I_1 e I_3 y dejar solo I_2 e I_4, porque "el hash es ligeramente más eficiente". ¿Qué consultas de (a) dejarían de acelerarse?

(c) Para un B+tree sobre $(\texttt{a}, \texttt{b})$, caracterice (con una condición sobre el orden de las hojas) qué predicados puede acelerar. Use su caracterización para explicar por qué el índice responde `a = 1 AND b >= 5` pero no `a >= 1 AND b = 5`.

Ejercicio 17.

Una relación tiene 20 000 tuplas y se quiere crear un B+tree sobre un atributo que es llave, de 40 bytes. Cada hoja guarda data entries (llave y puntero); cada puntero ocupa 10 bytes y cada página 1000 bytes. Se ignoran los encabezados y los punteros entre hojas.

(a) Un primer enfoque es ordenar las tuplas por esa llave e insertarlas una a una con el algoritmo de inserción (sin redistribución). ¿Qué problemas de costo y de uso de espacio tiene?

(b) Proponga un algoritmo de *carga masiva* que, a partir de las data entries ya ordenadas, construya el árbol de abajo hacia arriba escribiendo cada página una sola vez.

(c) Si su algoritmo llena cada página lo más posible, ¿cuántas data entries caben por hoja y cuántos hijos por directorio? ¿Cuántos niveles tiene el árbol y cuántos nodos hay en cada nivel?

(d) Repita (c) si las llaves del directorio se comprimen y ocupan en promedio 10 bytes (las hojas no cambian).

(e) Repita (c) si todas las páginas se llenan solo al 70 %. ¿Por qué podría convenir?

9. B+trees

Ejercicio 18.

Se parte con un B+tree de orden $d = 2$ vacío (hojas con 2 a 4 datos, directorios con 2 a 4 separadores) y se insertan, en este orden,

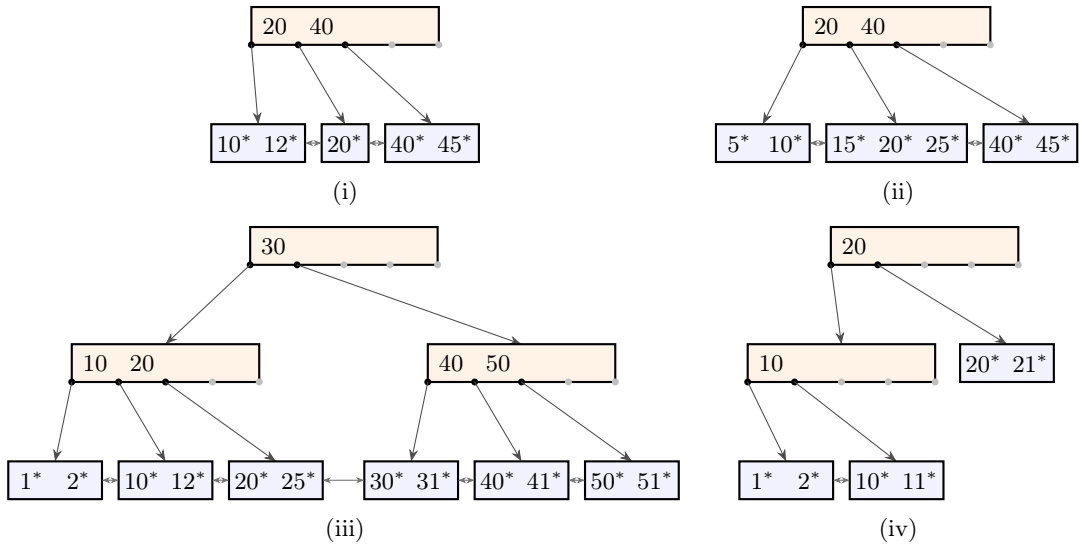
$$12,\ 7,\ 30,\ 18,\ 3,\ 25,\ 9,\ 21,\ 15,\ 40,\ 1,\ 27,\ 33,\ 10,\ 5,\ 22,\ 36,\ 14,\ 28,\ 19.$$

(a) Dibuje el árbol final sin redistribución. ¿Cuántos splits de hoja y cuántos de directorio hubo? ¿Cuál es la altura?

(b) Repita (a) usando redistribución: al desbordarse una hoja (o un directorio) se revisa primero el vecino derecho y luego el izquierdo (con el mismo padre); si alguno tiene espacio se le traspasa *un* dato (el mayor al derecho, el menor al izquierdo; en un directorio, pasando el separador por el padre) y solo si ninguno tiene espacio se hace split. ¿Cuántas páginas tiene ahora el árbol (hojas y directorios)?

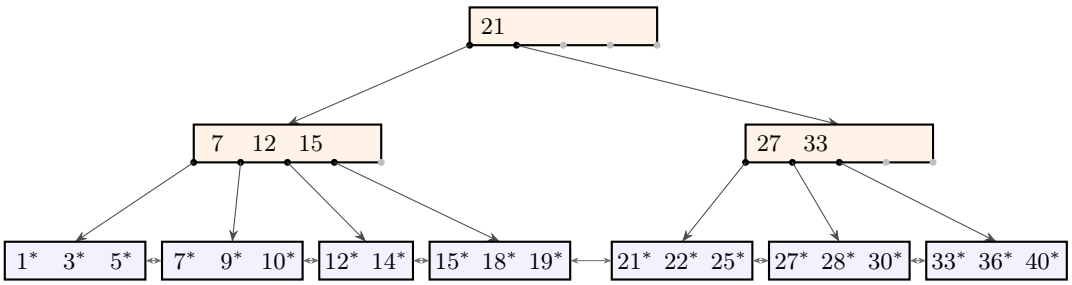
(c) En el árbol de (a), ¿cuántas páginas se leen para buscar 22? ¿Y para la consulta $10 \le A \le 25$ (se hace scan hasta leer la hoja donde aparece el primer dato mayor que 25)?

(d) Para cada uno de estos árboles de orden $d = 2$ indique si es un B+tree válido y, si no, qué regla viola:



Ejercicio 19.

Se usa el B+tree de orden $d = 2$ siguiente (el árbol de $12, 7, 30, \ldots, 19$ del ejercicio anterior, sin redistribución):

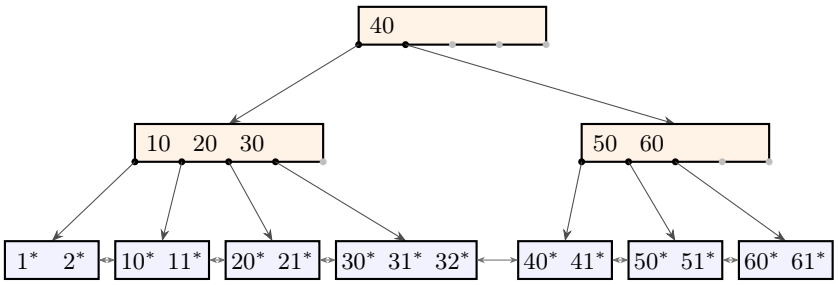


Eliminación: si la hoja queda con menos de d datos, se intenta redistribuir con el vecino izquierdo, luego con el derecho (con el mismo padre) y, si ninguno puede prestar, se hace unsplit con el izquierdo (o con el derecho si no hay izquierdo); lo mismo en los directorios. Los separadores no se modifican al eliminar sin rebalancear.

(a) Se eliminan, en este orden, 14, 19, 18, 15, 1, 3. Para cada eliminación diga si hay redistribución, unsplit (y de qué nodos) o nada, y dibuje el árbol después de eliminar 14 y el árbol final.

(b) ¿Qué eliminación reduce la altura del árbol y por qué? ¿Cuántas hojas y directorios tiene el árbol final?

(c) Sobre este otro árbol de orden $d = 2$, elimine el dato 60 y explique cada paso:



Ejercicio 20.

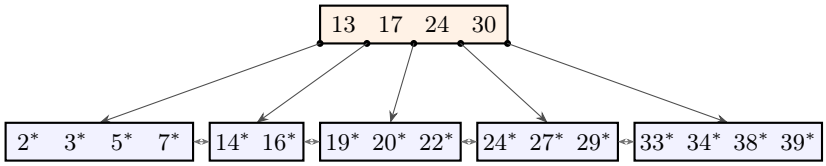
Considere B+trees de orden d (hojas con d a $2d$ datos; directorios, salvo la raíz, con d a $2d$ separadores; la raíz con al menos un separador). La altura $H \geq 1$ es el número de aristas de la raíz a una hoja, y suponga una data entry por dato.

(a) Demuestre que un B+tree de altura H tiene a lo más $2d(2d+1)^H$ datos y al menos $2d(d+1)^{H-1}$ datos. Deduzca cotas para la altura en función del número n de datos.

(b) Para $d = 2$ y $H = 2$, dibuje un B+tree con el mínimo y otro con el máximo número de datos, e indique cuántas hojas, cuántos directorios y cuántos datos tiene cada uno.

Ejercicio 21.

Considere el B+tree T de orden $d = 2$:



Usamos las convenciones del enunciado: sin redistribución al insertar; al eliminar, redistribuir con el vecino izquierdo, luego el derecho, y si no se puede, unsplit (solo con vecinos que tengan el mismo padre).

(a) Encuentre una lista de cinco valores a, b, c, d, e tal que, insertándolos en ese orden y luego eliminándolos en el orden inverso (e, d, c, b, a), se obtenga exactamente T (con los mismos separadores). Al menos una inserción debe provocar un split.

(b) Un compañero propone la lista $17, 18, 13, 15, 25$. ¿Vuelve al árbol original? Justifique mostrando el árbol final.

(c) Encuentre una lista de cinco valores tal que insertarlos y eliminarlos en orden inverso produzca un árbol *distinto* de T y de altura 2.

(d) Demuestre que, con $d = 2$, si insertar un valor k no provoca un split entonces eliminar k inmediatamente después devuelve el mismo árbol, y que si insertar k provoca un split de hoja, eliminar k inmediatamente después *nunca* devuelve el mismo árbol.

Ejercicio 22.

Considere B+trees de orden $d = 2$ que almacenan enteros.

(a) Para un n cualquiera, dé una secuencia de n inserciones en un B+tree vacío tal que (i) *sin* redistribución el índice use su tamaño máximo (todas las hojas, salvo a lo más una, a la mitad de su capacidad) y (ii) *con* redistribución en hojas y directorios la *misma* secuencia lo deje con su tamaño mínimo (todas las hojas, salvo a lo más dos, llenas). Explique por qué cumple lo pedido.

(b) Un B+tree de altura H está almacenado en disco y el buffer no guarda ninguna página entre operaciones. Se supone que una inserción lee la ruta de la raíz a la hoja, que un split de hoja obliga además a leer la hoja vecina derecha (para actualizar su puntero "anterior", pues las hojas están doblemente ligadas), que cada página nueva o modificada se escribe una vez y que la página de la raíz que desborda se reutiliza para una de sus mitades. ¿Cuál es el máximo de lecturas y de escrituras que puede costar *una* inserción? Dé un árbol y un dato que alcancen ese peor caso para $H = 2$.

Ejercicio 23.

Considere B+trees de orden $d = 2$ (hojas con 2 a 4 datos). Dé un ejemplo de cada caso, mostrando el árbol *antes* y *después* de la operación y explicando por qué su ejemplo cumple lo pedido.

(a) Un B+tree cuya altura cambia de 2 a 3 al insertar el valor 25. ¿Cuál es la menor cantidad de hojas que puede tener un árbol así?

(b) Un B+tree en el que eliminar el valor 25 produce una redistribución entre hojas.

(c) Un B+tree en el que eliminar el valor 25 produce un unsplit de dos hojas, sin que cambie la altura.

(d) Un B+tree en el que eliminar el valor 25 hace que la altura disminuya.

10. Hashing

Ejercicio 24.

Un índice de static hashing tiene $N = 4$ buckets con páginas de capacidad $B = 3$ (con páginas de overflow encadenadas) y $h(k) = k$ mód 4. Parte vacío y se insertan, en este orden, los datos $8, 12, 5, 16, 21, 4, 9, 20, 13, 3, 7, 24, 17, 28$.

(a) Dibuje el índice final (páginas principales y de overflow). ¿Cuántas páginas de overflow tiene y cuántos datos hay en cada bucket?

(b) ¿Cuántas páginas se leen para buscar 28, 17, 10 y 33? ¿Cuál es el costo promedio de buscar un dato que sí está en el índice?

(c) Repita (a) y (b) con $N = 5$ y $h(k) = k$ mód 5. ¿Cuál función funciona mejor con estos datos y por qué?

(d) Calcule $h(k) = (3k + 2)$ mód 7 para $k = 10, 15, 22, 31, 40, 53$. Con las llaves $10, 20, \ldots, 100$ y $h(k) = k$ mód N, ¿cuántos buckets distintos se usan si $N = 10$, $N = 8$ y $N = 7$? ¿Qué enseñanza deja?

(e) Una relación tiene 2 000 000 de tuplas y cada página guarda $B = 100$ tuplas. Con una buena función de hash (distribución uniforme), ¿cuántos buckets N se necesitan para que el costo de una operación sea aproximadamente 1? ¿Y si la relación crece a 20 000 000 de tuplas con el mismo N? ¿Qué costo tiene pasar de N a $2N$?

Ejercicio 25.

Un extendable hashing con buckets de capacidad $B = 2$ parte vacío (profundidad global 0 y un bucket de profundidad local 0). **Inserción:** como en la clase 15 (si tras un split el dato sigue sin

caber, se repite el algoritmo con el mismo dato). **Eliminación:** se elimina el dato de su bucket; si el bucket queda vacío y el bucket con el que se dividió por última vez (el que difiere solo en el bit de su profundidad local) tiene la misma profundidad local, se fusionan y la profundidad local baja en uno; luego, mientras todos los buckets tengan profundidad local menor que la global, el directorio se reduce a la mitad. Se insertan, en este orden, $12, 5, 3, 18, 6, 29, 10, 16, 20, 27$.

(a) Para cada inserción indique el bucket (entrada del directorio) y qué ocurre: (i) cabe, (ii) split sin doblar el directorio, (iii) split con doblamiento. Dibuje el índice tras insertar 3, 6, 10 y 20, y al final.

(b) ¿Cuántos splits y cuántos doblamientos hubo en total? ¿Cuántos buckets tiene el índice final y de qué tamaño es el directorio?

(c) Se eliminan, en este orden, $12, 16, 20, 10, 18, 5, 3$. Para cada eliminación indique si el bucket queda vacío, si hay merge, si baja la profundidad local y si baja la global. Dibuje el índice tras eliminar 16, tras eliminar 20, tras eliminar 18 y al final.

(d) Al eliminar 20 el bucket queda vacío y no se fusiona. Explique por qué.

(e) Sea un índice con $B = 2$ y un único bucket con $\{0, 2\}$. Muestre que, con esta regla, insertar 1 y luego eliminarlo provoca *cada vez* un doblamiento y una reducción del directorio. ¿Qué dice esto sobre el uso del merge, sabiendo que una inserción que dobla cuesta $O(D)$ con D el tamaño del directorio?

Ejercicio 26.

Una relación tiene 10^7 tuplas. Las páginas son de 4 KB, cada entrada del directorio (un puntero) ocupa 8 bytes y cada data entry (llave y puntero a la tupla) ocupa 16 bytes. El índice es un extendable hashing sobre un atributo; el directorio está almacenado en disco. Suponga una función de hash que reparte bien los datos.

(a) ¿Cuántas data entries caben en un bucket? ¿Cuántas entradas de directorio caben en una página?

(b) Dé una cota inferior para el número de buckets y, a partir de ella, para la profundidad global n y para el número de páginas del directorio.

(c) ¿Cuántas lecturas cuesta una búsqueda por igualdad si el directorio no está en memoria? ¿Y si lo está?

(d) Si el índice está justo en el límite de (b) y se produce un doblamiento, ¿cuántas páginas de directorio hay que escribir?

(e) Compare con un B+tree sobre el mismo atributo con hojas de 256 data entries y directorios de 256 hijos (páginas llenas): ¿cuántas lecturas cuesta una búsqueda por igualdad, con el índice unclustered (incluya la lectura de la tupla)? ¿Y con el extendable hashing?

(f) Con $B = 2$, ¿cuántos datos bastan para forzar profundidad global 5? Dé un ejemplo y demuestre que, en general, $B+1$ datos que coinciden en sus últimos t bits fuerzan una profundidad global de al menos $t + 1$. ¿Qué enseña esto sobre la cota de (b)?

Ejercicio 27.

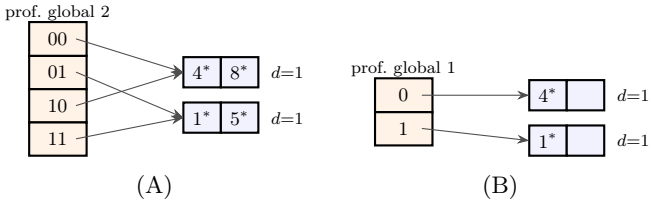
Un extendable hashing con $B = 2$ parte vacío. Con *solo inserciones* se cumplen dos propiedades:

(P1) la profundidad global es igual a la mayor profundidad local;

(P2) si dos buckets son hermanos (tienen la misma profundidad local $d \geq 1$ y sus entradas difieren solo en el bit d), entre los dos guardan al menos $B + 1$ datos.

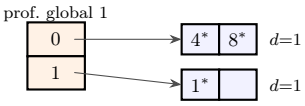
(a) Demuestre (P1) y (P2) para el algoritmo de inserción de la clase 15 (repitiendo el algoritmo si el dato sigue sin caber).

(b) Use (P1) y (P2) para mostrar que ninguno de estos dos estados válidos se puede obtener con solo inserciones:



(A) (B)

(c) El estado (C) cumple (P1) y (P2). Muestre una secuencia de inserciones que lo produce:



(d) ¿Se puede obtener (B) si se permiten eliminaciones (con la regla de merge y reducción del directorio del ejercicio 25)? ¿Y (A)?

Ejercicio 28.

Esquema: **Estudiantes**(eid, enombre, carrera), **Cursos**(ccod, cnombre, depto) e **Inscripciones**(ieid, iccod, nota, sem), con ieid referenciando a eid e iccod a ccod (eid y ccod son llaves primarias). Se usa la semántica con duplicados de SQL.

```
SELECT enombre, cnombre
FROM Estudiantes, Inscripciones, Cursos
WHERE eid = ieid AND iccod = ccod AND depto = 'DCC'
  AND nota >= 5 AND carrera = 'Ingenieria'
```

(a) Construya el plan lógico que entrega la regla de traducción de la clase 16 (producto de las relaciones del `FROM`, selección con el `WHERE` y proyección del `SELECT`).

(b) Convierta los productos en joins con condición y aplique push-selection de modo que cada selección quede lo más cerca posible de la relación a la que se refiere.

(c) Aplique push-projection: conserve en cada rama solo los atributos que se usan más arriba (en un join o en la proyección final).

(d) Escriba una consulta SQL, con subconsultas en el `FROM`, que corresponda al plan de (c).

(e) Un estudiante aprobó dos veces el mismo curso de `DCC` (dos inscripciones con nota ≥ 5). ¿Cuántas veces aparece el par (estudiante, curso) en el resultado de cada plan? ¿Cambia algo si el `SELECT` fuera `DISTINCT`? ¿Dónde pondría el δ?

Ejercicio 29.

(a) Para cada consulta sobre **Players**(id, name, age, team) y **Matches**(id, stadium, date) (age es entero) indique si es rechazada por el parser, por el preprocesador, o aceptada por ambos, y qué información del catálogo se usó cuando corresponda:

(I) `SELECT name FROM Players WHERE age >=`

(II) `SELECT nombre FROM Jugadores WHERE age = 30`

(III) `SELECT * FROM Players WHERE age = 'veinte'`

(IV) `SELECT id FROM Players, Matches WHERE age = 30`

(V) `SELECT * FROM Players WHERE name = 'Rodri' AND AND age = 30`

(VI) `SELECT * FROM Players, Matches WHERE age = 30`

(b) La consulta (vi) es aceptada por ambos. ¿Qué calcula? ¿Por qué un usuario probablemente no quería eso?

(c) Para cada decisión del procesador de consultas indique qué metadatos del catálogo necesita (información sobre las relaciones, estadísticas del contenido o información sobre los índices) y por qué: (i) usar un B+tree sobre age o recorrer **Players** para $\sigma_{\mathsf{age}=30}$; (ii) decidir si un δ puede eliminarse; (iii) decidir en qué orden hacer dos joins; (iv) rechazar la consulta (iii).

(d) Un ayudante dice: "el parser entrega un plan lógico". ¿Es correcto? Explique la diferencia entre el árbol de parsing y el plan lógico.

Ejercicio 30.

En cada ítem, R, S y T son relaciones con duplicados (los operadores $\cup$, $\cap$ y $\setminus$ tienen las multiplicidades de `UNION ALL`, `INTERSECT ALL` y `EXCEPT ALL`), con los mismos atributos cuando la operación lo exige. Decida si la igualdad vale para *toda* instancia. Si es falsa, dé una instancia mínima donde las dos expresiones difieran y, si existe, una condición razonable que la haga valer; si es verdadera, demuéstrela contando multiplicidades.

(a) $\sigma_\theta(R \setminus S) = \sigma_\theta(R) \setminus S$

(b) $\delta(R \cup S) = \delta(R) \cup \delta(S)$

(c) $R \cap (S \cup T) = (R \cap S) \cup (R \cap T)$

(d) $\pi_{\mathrm{att}(R)}(R \bowtie S) = R$

(e) $\delta(R \bowtie S) = \delta(R) \bowtie \delta(S)$

Ejercicio 31.

Esquema: **Pedidos**(pid, cid, total) con llave primaria pid y **Clientes**(cid, cnombre, ciudad) con llave primaria cid. Reglas de la clase 16: $\delta(R) = R$ si R tiene llave primaria o es resultado de un group-by (o de operadores sin duplicados); $\delta(\gamma_L(R)) = \gamma_L(R)$; $\gamma_L(\delta(R)) = \gamma_L(R)$ si L usa MAX o MIN; $\delta(\sigma_p(R)) = \sigma_p(\delta(R))$; $\delta(R \bowtie S) = \delta(R) \bowtie \delta(S)$.

Para cada consulta escriba el plan lógico, simplifique el δ cuando se pueda (diga qué regla usó y por qué se cumple su condición) y, si no se puede, dé una instancia donde el δ cambia el resultado.

(a) `SELECT DISTINCT cid FROM Pedidos`

(b) `SELECT DISTINCT cid, SUM(total) FROM Pedidos GROUP BY cid`

(c) `SELECT cid, MAX(total) FROM (SELECT DISTINCT * FROM Pedidos) GROUP BY cid`

(d) `SELECT cid, SUM(total) FROM (SELECT DISTINCT * FROM Pedidos) GROUP BY cid`

(e) `SELECT DISTINCT cnombre FROM Clientes`

(f) `SELECT DISTINCT cid, cnombre FROM Clientes`

(g) `SELECT DISTINCT cnombre, total FROM Clientes, Pedidos WHERE Clientes.cid = Pedidos.cid`

11. Problemas integradores

Ejercicio 32.

Sea $\mathbf{R}(A, B, C, D, E)$ con

$$\mathcal{F} = \{AE \to B,\ BC \to A,\ AB \to C,\ AE \to C,\ BCD \to A\}.$$

(a) Encuentre un conjunto minimal equivalente a $\mathcal{F}$. Justifique cada reducción de un lado izquierdo y cada dependencia eliminada con el cálculo de clausura correspondiente.

(b) Encuentre todas las llaves de $\mathbf{R}$. Sin examinar las dependencias una por una, explique por qué $\mathbf{R}$ está en 3NF.

(c) ¿Está $\mathbf{R}$ en BCNF? Dé todas las dependencias con un solo atributo a la derecha y lado izquierdo minimal que la violan.

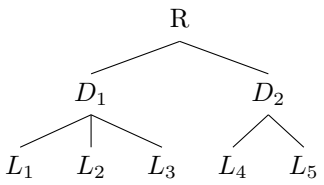
(d) Aplique `toBCNF` partiendo por cada dependencia del minimal de (a). Para cada descomposición obtenida decida si es sin pérdida y si preserva las dependencias. ¿Existe una descomposición de $\mathbf{R}$ en BCNF que preserve $\mathcal{F}$? Si existe, dé una y justifique que cada relación está en BCNF.

(e) Se agrega a $\mathcal{F}$ la dependencia $A \to C$, y llamamos $\mathcal{F}_1$ al nuevo conjunto. Recalcule el minimal y las llaves. Demuestre que ahora *ninguna* descomposición de $\mathbf{R}$ en relaciones en BCNF (obtenida con `toBCNF` o de cualquier otra forma, y aunque ni siquiera se exija que sea sin pérdida) preserva $\mathcal{F}_1$.

(f) Aplique a $\mathcal{F}_1$ el algoritmo de descomposición en 3NF. Muestre, con una instancia concreta, una anomalía de actualización en el resultado.

Ejercicio 33.

Un B+tree tiene la forma de la figura: una raíz R, dos directorios D_1 y D_2 y cinco hojas $L_1, \ldots, L_5$ (L_1, L_2, L_3 cuelgan de D_1 y L_4, L_5 de D_2).



Una búsqueda por valor que termina en la hoja L_i hace, en este orden, $\texttt{usar}(R)$, $\texttt{usar}(D)$ y $\texttt{usar}(L_i)$ (donde D es el directorio padre de L_i), con $\texttt{usar}(X)$ igual a $\texttt{pin}(X)$ seguido de $\texttt{unpin}(X, \texttt{false})$. Las búsquedas son las únicas operaciones sobre el buffer, que parte vacío y tiene N frames; se cuentan las lecturas a disco de una secuencia de m búsquedas.

(a) Con $N = 3$ y LRU, demuestre que desde la segunda búsqueda la página R siempre está en el buffer cuando se pide, y que cada búsqueda lee a lo más 2 páginas. Deduzca que m búsquedas leen a lo más $2m + 1$ páginas y dé una secuencia de m búsquedas que lo alcance.

(b) Con $N = 3$ y FIFO, demuestre que si una búsqueda lee 3 páginas, la siguiente lee a lo más 2. Deduzca que m búsquedas leen a lo más $\lceil 5m/2 \rceil$ páginas y dé una secuencia que lo alcance. Compare con (a) para la misma secuencia con $m = 6$.

(c) Con LRU, demuestre que el menor N para el cual el número total de lecturas queda acotado independientemente de m es $N = 8$. (Para $N = 8$ dé la cota; para $N \leq 7$ construya, para cada m, una secuencia con al menos m lecturas.)

(d) Con LRU y $N = 7$, demuestre que m búsquedas leen a lo más $m + 3$ páginas (para $m \geq 2$) y que la cota se alcanza.

Ejercicio 34.

Un B+tree de orden $d = 2$ (hojas con 2 a 4 datos y directorios con 2 a 4 separadores, salvo la raíz) parte vacío y se le insertan n valores distintos, *sin* redistribución y en un orden desconocido. Se sabe solo que los valores son $1, 2, \ldots, n$.

(a) Para $n = 12$, demuestre que el árbol final tiene altura 1 sin importar el orden de inserción.

(b) Para $n = 13$, demuestre que existe un orden con altura final 2 y que, *salvo ese*, no hay otro árbol final de altura 2: dibuje el único árbol posible.

(c) Para el árbol de (b), ¿cuáles valores pueden ser los *últimos* en insertarse? Justifique cada caso (los que pueden y los que no).

(d) Para el mismo árbol, ¿puede el 13 ser el *primer* valor insertado? ¿Y el 1? Dé en cada caso un orden completo que produzca el árbol, o demuestre que no existe.

(e) Dé un orden que produzca altura 1 para $n = 13$ y explique por qué los 13 valores en orden descendente no producen el árbol de (b).

Ejercicio 35.

En el extendable hashing de la clase 15, cuando se divide un bucket cuya profundidad local es igual a la profundidad global se *duplica* el directorio. Se propone una variante: en ese caso el directorio se *triplica*, y todo bucket que se divide se reparte en *tres* buckets. Los datos son enteros no negativos y los buckets tienen capacidad B.

(a) Defina la variante: la función h_n que se usa con profundidad global n, qué significa la profundidad local de un bucket, a cuántas entradas del directorio apunta un bucket, cómo se reparten los datos de un bucket que se divide y cómo se actualizan los punteros al triplicar y al dividir. Describa la búsqueda y la inserción (incluyendo el caso en que el dato nuevo sigue sin caber tras un split). ¿Cambia el costo de una búsqueda? ¿Y el de una inserción que triplica el directorio?

(b) Con $B = 2$ se insertan, en este orden, 12, 4, 7, 25, 16, 2, 11, 20. Para cada inserción indique cuántos splits y cuántos triplicamientos provoca. Al final, ¿cuál es la profundidad global, cuántos buckets hay y cuántas entradas tiene el directorio?

(c) Demuestre: (i) un bucket de profundidad local d es apuntado por exactamente 3^{n-d} entradas; (ii) el número de buckets siempre es impar; (iii) si hay N datos entonces $n \geq \lceil \log_3 \lceil N/B \rceil \rceil$, pero no existe una cota superior de n en función de N y B.

(d) Demuestre que ninguna de las dos versiones (duplicar o triplicar) es mejor que la otra en el tamaño del directorio: para $B = 2$ y cada $k \geq 1$, dé tres datos para los que la variante ternaria necesita 3^{k+1} entradas y la de la clase solo 2, y tres datos para los que la de la clase necesita 2^{k+1} y la ternaria solo 3.

(e) Alguien propone que, al triplicar el directorio, el bucket que se divide se reparta solo en *dos* buckets. ¿Qué propiedad de (c) deja de cumplirse y qué información adicional habría que guardar en cada bucket para que la búsqueda siga funcionando?

Ejercicio 36.

Esquema: **Clientes**(cid, cnombre, ciudad) con llave primaria cid y **Pedidos**(pid, pcid, total) con llave primaria pid y pcid referenciando a cid. Se trabaja con duplicados ($\cup$ es `UNION ALL`, π no elimina duplicados y δ los elimina). Sean $\theta_1 :$ cid = pcid, $\theta_2 :$ total $> 100 \vee$ ciudad = 'SCL', $\theta_3 :$ total ≤ 500. La consulta es

```
SELECT DISTINCT cnombre FROM Clientes, Pedidos
WHERE cid = pcid AND (total > 100 OR ciudad = 'SCL') AND total <= 500
```

Un compañero la reescribe así. Cada uno de los siguientes planes se obtiene del anterior por una regla que él cree válida.

$$P_0 = \delta\, \pi_{\text{cnombre}}\, \sigma_{\theta_1 \wedge \theta_2 \wedge \theta_3}(\textbf{Clientes} \times \textbf{Pedidos})$$

$$P_1 = \delta\, \pi_{\text{cnombre}}\, \sigma_{\theta_2}\big(\sigma_{\theta_3}(\textbf{Clientes} \bowtie_{\theta_1} \textbf{Pedidos})\big)$$

$$P_2 = \delta\, \pi_{\text{cnombre}}\, \sigma_{\theta_2}\big(\textbf{Clientes} \bowtie_{\theta_1} \sigma_{\theta_3}(\textbf{Pedidos})\big)$$

$$P_3 = \delta\, \pi_{\text{cnombre}}\Big(\big(\textbf{Clientes} \bowtie_{\theta_1} \sigma_{\text{total}>100}\sigma_{\theta_3}(\textbf{Pedidos})\big) \cup \big(\sigma_{\text{ciudad}=\text{'SCL'}}(\textbf{Clientes}) \bowtie_{\theta_1} \sigma_{\theta_3}(\textbf{Pedidos})\big)\Big)$$

$$P_4 = \delta\Big(\pi_{\text{cnombre}}\big(\textbf{Clientes} \bowtie_{\theta_1} \pi_{\text{pcid}}\sigma_{\text{total}>100}\sigma_{\theta_3}(\textbf{Pedidos})\big) \cup \pi_{\text{cnombre}}\big(\pi_{\text{cid,cnombre}}\sigma_{\text{ciudad}=\text{'SCL'}}(\textbf{Clientes}) \bowtie_{\theta_1} \pi_{\text{pcid}}\sigma_{\theta_3}(\textbf{Pedidos})\big)\Big)$$

$P_5 = P_4$ sin el δ externo, "porque cid es llave primaria de **Clientes** y no hay duplicados"

$P_6 = P_4$ cambiando en la primera rama **Clientes** por $\pi_{\text{cnombre}}(\textbf{Clientes})$ "para proyectar lo antes posible"

(a) Para cada paso $P_0 \to P_1$, $P_1 \to P_2$, $P_2 \to P_3$, $P_3 \to P_4$, $P_4 \to P_5$ y $P_4 \to P_6$ decida si la reescritura es: (i) correcta (los dos planes dan el mismo resultado, con duplicados, para toda instancia); (ii) incorrecta; o (iii) mal definida. Para cada paso nombre la regla de la clase que se aplicó (o debió aplicarse) y justifique.

(b) En los pasos correctos, distinga cuáles lo son aun *sin* el δ externo (es decir, para las expresiones interiores como multiconjuntos) y cuáles lo son *solo gracias* a él. Para estos últimos dé una instancia donde los multiconjuntos interiores difieren.

(c) Para los pasos incorrectos o mal definidos dé una instancia (o explique por qué no se puede evaluar) y corrija el paso cuando sea posible.

(d) Si la consulta no tuviera `DISTINCT` (se elimina el δ de todos los planes), ¿cuáles de los pasos de P_0 a P_4 siguen siendo correctos?